Programiz

C++ Online Compiler

Programiz PRO

Single inheritance

main.cpp

```cpp
#include <iostream>
using namespace std;

class Person {
public:
    void show() {
        cout << "I am a person\n";
    }
};

class Student : public Person {
public:
    void study() {
        cout << "I study C++\n";
    }
};

int main() {
    Student s;
    s.show();
    s.study();
    return 0;
}
```

Run

main.c... Output

```
I am a person
I study C++


=== Code Execution Successful ===
```

Programiz
C++ Online Compiler

Programiz PRO

MULTILEVEL INHERITANCE

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() {
        cout << "Eating\n";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Barking\n";
    }
};

class Puppy : public Dog {
public:
    void weep() {
        cout << "Weeping\n";
    }
};

int main() {
    Puppy p;
    p.eat();
    p.bark();
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```
using namespace std;

class Animal {
public:
    void eat() {
        cout << "Eating\n";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Barking\n";
    }
};

class Puppy : public Dog {
public:
    void weep() {
        cout << "Weeping\n";
    }
};

int main() {
    Puppy p;
    p.eat();
    p.bark();
    p.weep();
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```
Eating
Barking
Weeping


=== Code Execution Successful ===
```

Programiz
C++ Online Compiler

Programiz PRO

main.c

Output

Hierarchical Inheritance

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    void draw() {
        cout << "Drawing shape\n";
    }
};

class Circle : public Shape {
};

class Square : public Shape {
};

int main() {
    Circle c;
    Square s;
    c.draw();
    s.draw();
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```
Drawing shape
Drawing shape


=== Code Execution Successful ===
```

Programiz
C++ Online Compiler

Programiz PRO

Hybrid Inheritance

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void display() {
        cout << "Class A\n";
    }
};

class B : public A {};
class C : public A {};

class D : public B, public C {
};

int main() {
    D obj;
    obj.B::display(); // avoid ambiguity
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c...

Output

```
Class A


=== Code Execution Successful ===
```

Programiz
C++ Online Compiler

Programiz PRO

VIRTUAL FUNCTION

main.c... Output

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {
        cout << "Base class\n";
    }
};

class Derived : public Base {
public:
    void show() {
        cout << "Derived class\n";
    }
};

int main() {
    Base* b;
    Derived d;
    b = &d;
    b->show(); // calls derived function
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c...

Output

```
Derived class


=== Code Execution Successful ===
```

Programiz

C++ Online Compiler

Programiz PRO

DYNAMIC BINDING

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() {
        cout << "Animal sound\n";
    }
};

class Dog : public Animal {
public:
    void sound() {
        cout << "Dog barks\n";
    }
};

int main() {
    Animal* a;
    Dog d;
    a = &d;
    a->sound(); // dynamic binding
}
```

Programiz
C++ Online Compiler

Programiz PRO

main.c...

Output

```
Dog barks


=== Code Execution Successful ===
```

Programiz

C++ Online Compiler

Programiz PRO

main.cpp | Output

THIS POINTER

```cpp
#include <iostream>
using namespace std;

class Student {
    int age;
public:
    void setAge(int age) {
        this->age = age;
    }

    void display() {
        cout << "Age: " << this->age;
    }
};

int main() {
    Student s;
    s.setAge(20);
    s.display();
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```
Age: 20

=== Code Execution Successful ===
```

Programiz

C++ Online Compiler

Programiz PRO

Abstract Class Example 1

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0; // pure virtual
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing Circle\n";
    }
};

int main() {
    Circle c;
    c.draw();
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```
Drawing Circle


=== Code Execution Successful ===
```

Programiz
C++ Online Compiler

Programiz PRO

Abstract Class Example 2

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    virtual void start() = 0;
};

class Car : public Vehicle {
public:
    void start() {
        cout << "Car starts with key\n";
    }
};

class Bike : public Vehicle {
public:
    void start() {
        cout << "Bike starts with button\n";
    }
};

int main() {
    Car c;
```

```
using namespace std;

class Vehicle {
public:
    virtual void start() = 0;
};

class Car : public Vehicle {
public:
    void start() {
        cout << "Car starts with key\n";
    }
};

class Bike : public Vehicle {
public:
    void start() {
        cout << "Bike starts with button\n";
    }
};

int main() {
    Car c;
    Bike b;
    c.start();
    b.start();
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```
Car starts with key
Bike starts with button


=== Code Execution Successful ===
```

Programiz

C++ Online Compiler

Programiz PRO

VIRTUAL FUNCTION example 2

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() {
        cout << "Area of shape\n";
    }
};

class Rectangle : public Shape {
public:
    void area() {
        cout << "Area of Rectangle = length × breadth\n";
    }
};

class Circle : public Shape {
public:
    void area() {
        cout << "Area of Circle = π × r × r\n";
    }
};

int main() {
    Shape* s;
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```cpp
        cout << "Area of Rectangle = length × breadth\n";
    }
};

class Circle : public Shape {
public:
    void area() {
        cout << "Area of Circle = π × r × r\n";
    }
};

int main() {
    Shape* s;
    Rectangle r;
    Circle c;

    s = &r;
    s->area();   // calls Rectangle function

    s = &c;
    s->area();   // calls Circle function

    return 0;
}
```

Run

Programiz
C++ Online Compiler

Programiz PRO

main.c... Output

```
Area of Rectangle = length × breadth
Area of Circle = π × r × r


=== Code Execution Successful ===
```